

APOLLO INSTITUTE OF COMPUTER APPLICATION
BACHELOR OF COMPUTER APPLICATION
SEMESTER 03
Object Oriented Programming with Java (BC03001021)
Assignment 1
UNIT 1: Introduction to Java

1 Mark Questions- *Winter 2025*

1. What is a class in Java?

A **class** in Java is a blueprint or template used to create objects. It defines the **data members (variables)** and **methods (functions)** that an object can have.

2. What are subclasses and superclasses?

A **superclass** is a class whose properties and methods are inherited by another class.

A **subclass** is the class that inherits from the superclass.

Example: If Animal is a superclass and Dog extends Animal, then Dog is the subclass.

3. What is the advantage of Java over C++?

Java has several advantages over C++, such as:

- Platform independence ("Write Once, Run Anywhere")
- Automatic garbage collection
- No direct pointer manipulation
- Built-in support for multithreading
- Simpler and safer object-oriented programming

4. What is the use of separators in a Java program?

Separators are symbols used to define the structure of a Java program. Examples include:

- `;` — separates statements
- `,` — separates variables or arguments
- `{ }` — defines a block of code
- `( )` — encloses method parameters or expressions
- `[ ]` — represents arrays

5. What is polymorphism?

Polymorphism means "many forms." In Java, it allows the same method or object to behave differently in different situations.

The two main types are:

- **Compile-time polymorphism** — method overloading
- **Run-time polymorphism** — method overriding

1 Mark Questions- Summer 2026

6. Which class in Java is the parent of all classes?

Answer: Object class.

7. Java programs are compiled into _____ which are executed by the JVM.

Answer: Bytecode

8. _____ keyword is used to refer to the current object in Java.

Answer: this

9. Give full form of JDK.

Answer: Java Development Kit

10. Define term JRE.

Answer: JRE (Java Runtime Environment) is a software environment used to **run Java programs**. It contains the JVM and required Java class libraries.

11. Define term static keyword.

Answer: The static keyword is used to define **class-level members** that belong to the class rather than individual objects. Static variables and methods can be accessed using the class name.

7 Mark Questions- Winter 2025

1. Compare procedure-oriented and object-oriented programming.

Key Point	Procedure-Oriented Programming (POP)	Object-Oriented Programming (OOP)
1. Definition	POP is a programming approach in which a program is divided into functions or procedures .	OOP is a programming approach in which a program is organized using objects and classes .
2. Main Focus	It mainly focuses on functions and the sequence of operations performed by the program.	It mainly focuses on data and the objects that operate on that data .
3. Approach	It follows a top-down approach , where a large problem is divided into smaller functions.	It generally follows a bottom-up approach , where objects and classes are designed first and then combined to form the program.
4. Data Security	Data is generally less secure because it can be accessed by different functions.	Data is more secure because it can be hidden inside objects using encapsulation and access modifiers.
5. Program Division	The program is divided into functions or procedures .	The program is divided into classes and objects .
6. Data and Functions	Data and functions are usually treated as separate entities .	Data and methods are combined together inside a class or object.
7. Reusability	Code reusability is limited .	Code can be reused easily through inheritance, classes, and methods .

Key Point	Procedure-Oriented Programming (POP)	Object-Oriented Programming (OOP)
8. Inheritance	Inheritance is not supported as a basic feature.	Inheritance is supported , allowing one class to acquire properties and methods of another class.
9. Polymorphism	Polymorphism is not a major feature of POP.	Polymorphism is an important feature of OOP, allowing the same method to behave differently in different situations.
10. Modularity	Modularity is achieved mainly by dividing the program into functions.	Modularity is achieved by creating classes and objects , making large programs easier to manage.
11. Maintenance	Large programs can be more difficult to maintain because changes in one part may affect other functions.	Large programs are easier to maintain because data and functionality are organized into separate classes and objects.
12. Examples	C, Pascal, FORTRAN	Java, C++, C#

2. Explain the features of Java

Java is a **high-level, object-oriented, secure, and platform-independent programming language**. The important features of Java are:

1. Simple

Java is easy to learn and understand. It removes complicated features of C++, such as **pointers, operator overloading, and multiple inheritance through classes**.

2. Object-Oriented

Java follows the **object-oriented programming (OOP)** concept. It supports **classes, objects, inheritance, polymorphism, encapsulation, and abstraction**.

3. Platform Independent

Java programs can run on different operating systems without changing the source code. Java follows the principle **“Write Once, Run Anywhere.”**

Java source code is converted into **bytecode**, which is executed by the JVM.

4. Portable

Java programs are portable because Java bytecode can be executed on any system that has a compatible **Java Virtual Machine (JVM)**.

5. Secure

Java provides a secure environment for running programs. It does not support direct pointer manipulation and provides features such as **bytecode verification and access control**.

6. Robust

Java is a robust language because it provides **strong memory management, exception handling, and automatic garbage collection**. These features help reduce errors and program crashes.

7. Architecture Neutral

Java bytecode is not dependent on a particular computer architecture. The same bytecode can run on different processors and operating systems through the JVM.

8. Multithreaded

Java supports **multithreading**, which allows multiple tasks to execute concurrently. This is useful for applications that need to perform several operations at the same time.

9. High Performance

Java provides good performance because its **JIT (Just-In-Time) compiler** converts frequently executed bytecode into native machine code during execution.

10. Distributed

Java supports the development of **distributed applications**. It provides networking features that allow programs to communicate and work across different computers.

11. Dynamic

Java is designed to adapt to a changing environment. Classes can be **loaded dynamically at runtime**, making Java suitable for applications that need flexibility.

12. Automatic Garbage Collection

Java automatically removes objects that are no longer being used through **garbage collection**. This reduces the need for programmers to manually manage memory.

3. Explain How Java achieves Encapsulation

Definition

Encapsulation is the process of **wrapping data (variables) and methods together inside a class** and restricting direct access to the data.

Java achieves encapsulation mainly through **classes, private variables, and public methods**.

How Java Achieves Encapsulation

1. Class:

Java combines data members and methods inside a class.

2. Private Data Members:

Variables are declared using the `private` keyword. This prevents direct access from outside the class.

3. Getter Method:

A getter method is used to **access or read** private data.

4. Setter Method:

A setter method is used to **set or modify** private data.

5. Controlled Access:

The programmer can control how data is accessed or changed through methods.

Simple Example

```
class Student {  
  
    private int marks;  
  
    public void setMarks(int m) {  
  
        marks = m;  
  
    }  
  
    public int getMarks() {  
  
        return marks;  
  
    }  
  
}
```

Explanation

- `marks` is declared as **private**, so it cannot be accessed directly outside the class.
- `setMarks()` is used to **set the marks**.
- `getMarks()` is used to **get the marks**.
- Thus, the data is protected and accessed in a controlled manner.

Advantages of Encapsulation

- Provides **data hiding**.
- Prevents **direct access** to data.
- Provides **controlled access**.
- Improves **data security**.

- Makes the program easier to **maintain and modify**.

Easy to remember:

Encapsulation = Private Data + Getter + Setter + Controlled Access.

4. Explain the difference between compile-time and runtime polymorphism.

Polymorphism means “**one name, many forms.**” It allows the same method name to perform different operations in different situations.

Java provides two types of polymorphism: **Compile-Time Polymorphism** and **Runtime Polymorphism**.

Basis	Compile-Time Polymorphism	Runtime Polymorphism
1. Definition	The method to be executed is determined during compilation .	The method to be executed is determined during program execution .
2. Also known as	Static Polymorphism or Early Binding	Dynamic Polymorphism or Late Binding
3. Achieved by	Achieved using Method Overloading .	Achieved using Method Overriding .
4. Inheritance	Inheritance is not required .	Inheritance is required .
5. Method parameters	Methods must have the same name but different parameters .	Parent and child classes have methods with the same name and same parameters .
6. Decision time	Method selection takes place at compile time .	Method selection takes place at runtime .
7. Binding	Uses early/static binding .	Uses late/dynamic binding .
8. Flexibility	It provides comparatively less flexibility .	It provides greater flexibility because the actual object determines the method.
9. Performance	Generally faster because the method is known during compilation.	Generally slightly slower because the method is determined during execution.
10. Example	add(int, int) and add(int, int, int)	Animal a = new Dog(); a.sound();

Example of Compile-Time Polymorphism – Method Overloading

```
class Calculator {

    int add(int a, int b) {

        return a + b;

    }

    int add(int a, int b, int c) {

        return a + b + c;

    }

}
```

Here, both methods have the same name add() but different numbers of parameters. The compiler decides which method should be called.

Example of Runtime Polymorphism – Method Overriding

```
class Animal {  
  
    void sound() {  
  
        System.out.println("Animal makes sound");  
  
    }  
}  
  
class Dog extends Animal {  
  
    void sound() {  
  
        System.out.println("Dog barks");  
  
    }  
}  
  
class Main {  
  
    public static void main(String[] args) {  
  
        Animal a = new Dog();  
  
        a.sound();  
  
    }  
}
```

Output:

Dog barks

Here, a is a reference of the Animal class, but it refers to a Dog object. Therefore, the Dog class's sound() method is executed at **runtime**.

Easy way to remember

Compile-Time → Overloading → Different parameters → Compiler decides

Runtime → Overriding → Parent-Child → Object decides

7 Mark Questions- Summer 2026

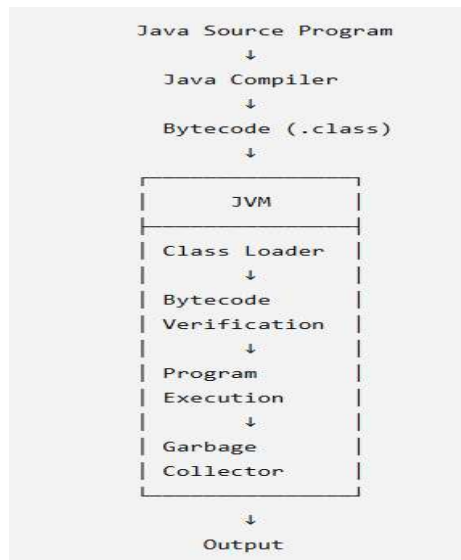
1. Explain JVM in detail.

JVM (Java Virtual Machine) is a part of the **Java Runtime Environment (JRE)**. It is a virtual machine that executes **Java bytecode**. JVM makes Java **platform independent**, following the principle “**Write Once, Run Anywhere.**”

The execution process of a Java program is:

Java Source Code → Compiler → Bytecode → JVM → Machine Code → Output

Simple JVM Flow



Main Components of JVM

1. Class Loader

The **Class Loader** is responsible for loading Java .class files (bytecode) into the JVM memory.

Its main functions are:

- Loads the required class files.
- Loads classes when they are needed during program execution.
- Helps JVM prepare the classes for execution.

Example: When we run Student.class, the Class Loader loads the Student class into the JVM.

2. Bytecode Verification

Bytecode Verification checks the bytecode before it is executed.

It ensures that:

- The bytecode is valid and follows Java rules.
- It does not perform illegal operations.
- It does not access memory incorrectly.
- The program follows Java's security restrictions.

Thus, bytecode verification helps make Java **secure and reliable**.

3. Program Execution

The JVM is responsible for **executing the Java bytecode**.

The **Execution Engine** performs this task. It uses:

- **Interpreter:** Executes bytecode instruction by instruction.
- **JIT Compiler:** Converts frequently used bytecode into machine code to improve execution speed.

Thus, JVM converts the platform-independent bytecode into instructions that the particular computer can understand.

4. Garbage Collector

The **Garbage Collector (GC)** is responsible for **automatic memory management**.

It:

- Identifies objects that are no longer being used.
- Removes those unused objects from memory.
- Frees the memory so that it can be reused.

This means the programmer does **not need to manually delete objects**.

Example:

```
Student s = new Student();
```

```
s = null;
```

After `s = null`, the previously created `Student` object may become unused. The **Garbage Collector** can eventually remove it from memory.

Advantages/Importance of JVM

- 1. Platform independence:** The same bytecode can run on different operating systems with a compatible JVM.
- 2. Memory management:** JVM automatically manages memory.
- 3. Security:** JVM provides a controlled environment for executing bytecode.
- 4. Garbage collection:** Unused objects are automatically removed.
- 5. Exception handling:** JVM supports Java's exception-handling mechanism.
- 6. Multithreading:** JVM provides support for multiple threads.

2. Explain Program Structure of Java.

A Java program follows a specific structure that makes the program easy to understand, compile, and execute. A basic Java program generally contains **package declaration, import statements, class declaration, variables, methods, and the main() method.**

Basic Structure of a Java Program

// 1. Package declaration

```
package mypackage;
```

// 2. Import statement

```
import java.util.Scanner;
```

// 3. Class declaration

```
public class Demo {
```

// 4. Data member

```
int number;
```

// 5. Method

```
void display() {
```

```
    System.out.println("Hello Java");
```

```
}
```

// 6. Main method

```
public static void main(String[] args) {
```

```
// 7. Creating object
```

```
Demo obj = new Demo();
```

```
// 8. Calling method
```

```
obj.display();
```

```
}
```

```
}
```

Components of Java Program Structure

1. Package Declaration

A package is used to **group related classes and interfaces**.

```
package mypackage;
```

It is optional and generally written at the beginning of the program.

2. Import Statement

The import statement is used to use classes and interfaces from other packages.

```
import java.util.Scanner;
```

It is also optional.

3. Class Declaration

Every Java program is written inside a **class**.

```
public class Demo {
```

```
}
```

The class contains variables and methods.

4. Data Members

Variables declared inside a class are called **data members** or instance variables.

```
int number;
```

They store data used by the program.

5. Methods

A method contains a set of statements that performs a particular task.

```
void display() {  
    System.out.println("Hello Java");  
}
```

6. Main Method

The main() method is the **starting point of execution** for a standard Java application.

```
public static void main(String[] args)
```

- public → accessible to JVM.
- static → JVM can call it without creating an object.
- void → does not return a value.
- main → name recognized as the entry point.
- String[] args → stores command-line arguments.

7. Object Creation

An object is created using the new keyword.

```
Demo obj = new Demo();
```

8. Method Calling

The object can be used to call the methods of the class.

```
obj.display();
```

Java Program Execution

Java Source File (.java)

↓

Compiler

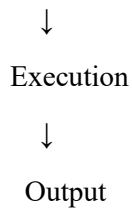
javac

↓

Bytecode (.class)

↓

JVM



OTHER MOST IMPORTANT QUESTION – 7 MARK

1. Differentiate between JDK, JRE and JVM.

Java has three important components: **JDK, JRE, and JVM**. They are related to each other but perform different functions.

Full Forms

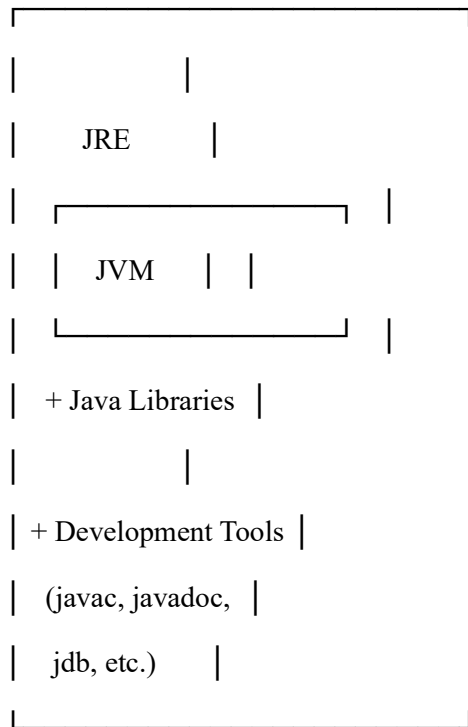
- **JDK** → Java Development Kit
- **JRE** → Java Runtime Environment
- **JVM** → Java Virtual Machine

Difference Between JDK, JRE and JVM

Basis	JDK	JRE	JVM
1. Full Form	Java Development Kit	Java Runtime Environment	Java Virtual Machine
2. Purpose	Used to develop, compile, and run Java programs.	Used to run Java applications .	Used to execute Java bytecode .
3. Contains	JRE + Development Tools	JVM + Java Libraries	Execution environment for bytecode
4. Compiler	Contains the Java compiler <code>javac</code> .	Does not contain the Java compiler.	Does not contain the Java compiler.
5. Development	Required for Java program development .	Not sufficient for development.	Not used directly for program development.
6. Execution	Can run Java programs.	Can run Java programs.	Executes Java bytecode.
7. Main Components	JRE, compiler, debugger and other development tools.	JVM and Java class libraries.	Class Loader, Execution Engine, memory management, etc.
8. Users	Mainly used by Java developers .	Used by users who only need to run Java applications .	Works behind the scenes to execute Java programs.
9. Example	<code>javac</code> , <code>java</code> , <code>javadoc</code> , <code>jdb</code>	JVM + Java libraries	Interpreter/JIT and runtime facilities
10. Relationship	JDK contains JRE.	JRE contains JVM.	JVM is inside JRE.

Relationship Between JDK, JRE and JVM

JDK



1. JDK – Java Development Kit

JDK is a complete package used to **develop Java applications**.

It contains:

- **JRE**
- Java compiler (javac)
- Debugging tools
- Documentation tools
- Other development utilities

For example:

```
javac Test.java
```

The javac compiler converts the Java source code into **bytecode**.

2. JRE – Java Runtime Environment

JRE provides the environment required to **run Java applications**.

It contains:

- **JVM**
- Java class libraries

- Supporting runtime files

JRE does **not** contain the Java compiler, so it is not sufficient for developing Java programs.

3. JVM – Java Virtual Machine

JVM is responsible for **executing Java bytecode**.

- Loads bytecode.
- Verifies bytecode.
- Executes the program.
- Manages memory.
- Performs garbage collection.

JVM is the main reason Java is **platform independent**.

Easy Way to Remember

JDK = Develop + Run

JRE = Run

JVM = Execute

Or remember:

JDK contains JRE,

and JRE contains JVM.

2. Explain Java Bytecode and how it makes Java platform independent.

What is Java Bytecode?

Java Bytecode is the **intermediate code generated by the Java compiler** after compiling a Java source program. It is stored in a file with the **.class extension**.

When a Java program is compiled using the javac command, the source code (.java) is converted into bytecode (.class).

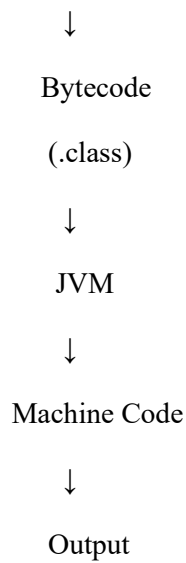
Java Source Code

(.java)

↓

Java Compiler

(javac)



Example

Suppose we have:

```
class Hello {  
  
    public static void main(String[] args) {  
  
        System.out.println("Hello Java");  
  
    }  
  
}
```

When we compile it:

```
javac Hello.java
```

The compiler generates:

Hello.class

The Hello.class file contains **Java bytecode**.

How Bytecode Makes Java Platform Independent

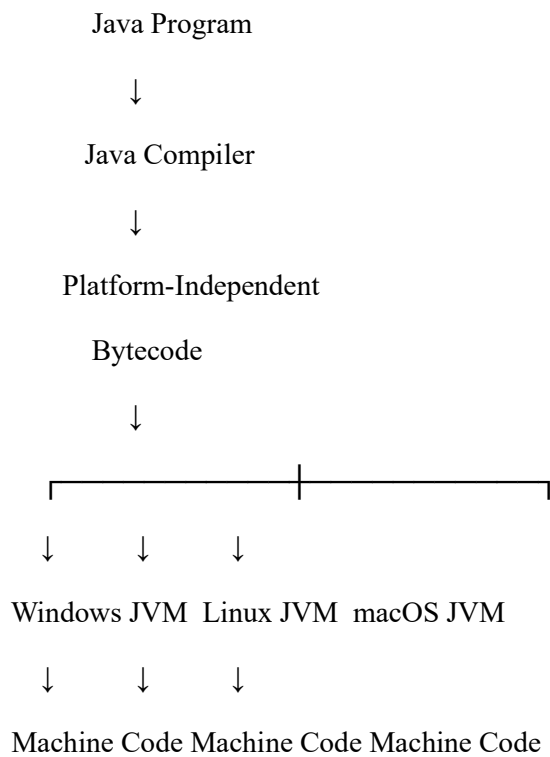
Java follows the principle:

“Write Once, Run Anywhere (WORA)”

The Java compiler does **not directly convert Java source code into machine code for a specific operating system**. Instead, it converts the source code into **platform-independent bytecode**.

The bytecode can run on different operating systems through their respective JVM.

For example:



Explanation

1. **Source code is written once** in Java.
2. The Java compiler converts it into **bytecode**.
3. Bytecode is **not specific to any operating system or processor**.
4. Each operating system has its own **JVM**.
5. The JVM converts/executes the bytecode according to the particular platform.
6. Therefore, the **same .class file can run on different platforms** without changing the Java source code.

Example

The same Hello.class file can be executed on:

- Windows
- Linux
- macOS

Important Point

Java is platform independent, but JVM is platform dependent.

This is because different operating systems require different JVM implementations, while the **bytecode remains the same**.

3. Explain classes and objects in Java with a suitable example.

Class

A **class** is a blueprint or template for creating objects in Java. It defines the **properties (data members)** and **behaviors (methods)** of objects.

A class can contain:

- Data members (variables)
- Methods
- Constructors
- Other members

Syntax:

```
class ClassName {  
  
    // data members  
  
    // methods  
  
}
```

Object

An **object** is an **instance of a class**. It represents a particular entity and has its own data and behavior.

An object is created using the **new keyword**.

Syntax:

```
ClassName objectName = new ClassName();
```

Short Example

```
class Student {  
  
    String name;  
  
  
    void display() {  
        System.out.println(name);  
    }  
}  
  
class Main {  
  
    public static void main(String[] args) {  
        Student s = new Student();  
        s.name = "Rahul";  
        s.display();  
    }  
}
```

Output:

Rahul

Explanation of Example

- Student is the **class**.
- name is a **data member** that stores the student's name.
- display() is a **method** that displays the name.
- s is an **object** of the Student class.
- new Student() creates a new object.
- s.name = "Rahul" assigns a value to the object's data member.
- s.display() calls the display() method using the object.

Difference Between Class and Object

Class	Object
A class is a blueprint or template.	An object is an instance of a class.
It defines properties and behaviors.	It represents a specific entity.
It is a logical concept.	It is a runtime entity.
It does not represent a particular object.	It contains its own data/state.
Example: Student	Example: s

4. Explain abstraction and encapsulation in Java with suitable examples.

1. Abstraction

Abstraction is the process of **hiding implementation details and showing only the essential features** to the user.

It focuses on **what an object does** rather than **how it does it**.

Java achieves abstraction mainly using:

- **Abstract classes**
- **Interfaces**

Example of Abstraction

```
abstract class Animal {  
    abstract void sound();  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

Explanation

- Animal is an **abstract class**.
- sound() is an **abstract method** without a body.
- Dog provides the implementation of sound().
- The user only needs to know that the Dog produces a sound; the implementation is provided by the Dog class.

Real-life

example:

When we use an ATM, we select options such as **withdraw money** or **check balance**. We do not need to know the internal processing of the bank system. This is abstraction.

2. Encapsulation

Encapsulation is the process of **wrapping data and methods together inside a class** and restricting direct access to the data.

It is mainly achieved using:

- **Classes**
- **Private data members**
- **Public getter and setter methods**

Example of Encapsulation

```
class Student {  
  
    private int marks;  
  
    public void setMarks(int m) {  
        marks = m;  
    }  
  
    public int getMarks() {  
        return marks;  
    }  
}
```

Explanation

- marks is declared as **private**, so it cannot be directly accessed from outside the class.
- setMarks() is used to **set or modify** the value.

- getMarks() is used to **access** the value.
- Thus, access to the data is controlled.

Real-life

example:

A bank account protects its balance from direct modification. The balance can be changed through controlled operations such as deposit or withdrawal.

Difference Between Abstraction and Encapsulation

Abstraction	Encapsulation
Hides implementation details .	Hides/protects data from direct access.
Focuses on what to do .	Focuses on how to protect data .
Achieved using abstract classes and interfaces .	Achieved using classes, private variables, getters and setters .
Helps reduce complexity.	Helps provide data security and controlled access.
Example: ATM interface.	Example: Private bank account balance.

1 MARK

Possible Question	Answer to remember
Which class is parent of all Java classes?	Object
Java source code is compiled into?	Bytecode
Bytecode is stored in which file?	.class file
Which component executes bytecode?	JVM
Full form of JVM?	Java Virtual Machine
Full form of JDK?	Java Development Kit
Full form of JRE?	Java Runtime Environment
Which keyword is used for inheritance?	extends
Which keyword is used to define a class?	class
Which keyword creates a subclass relationship?	extends
Which OOP feature allows one interface/method to have different forms?	Polymorphism
Which feature hides implementation details?	Abstraction
Which feature bundles data and methods?	Encapsulation
Java is platform _____.	Independent
Java source file extension?	.java
Java bytecode file extension?	.class
Which method is the starting point of a Java application?	main()
Standard main method signature?	public static void main(String[] args)